

Assignment: 06

Q22
psum(13)
fun(3)

void fun(int n)

if(n==0)
return;

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

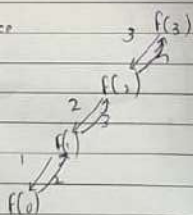
void fun(int n)
if(n==0)
return;

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

Recursive Tree



c/p: 3

2

1

2

3

4

(3 2 1 2 3)

Call Stack

void fun(int n)
if(n==0)
return;

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

fun(n-1);

S.O.P(n+" ");

fun(n-1);

Q23
psum(1)
fun(2)

void fun(int n)

if(n==0)
return;

S.O.P(x+" ");

fun(n-1);

S.O.P(y+" ");

fun(n-1);

void fun(int n)

if(n==0)
return;

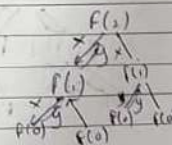
S.O.P(x+" ");

fun(n-1);

S.O.P(y+" ");

fun(n-1);

Recursive Tree



o/p: x

x

y

y

x

y

[x x y y x y]

f(2) - pop

f(1) - pop

f(1) - pop

f(0) - pop

f(0) - pop

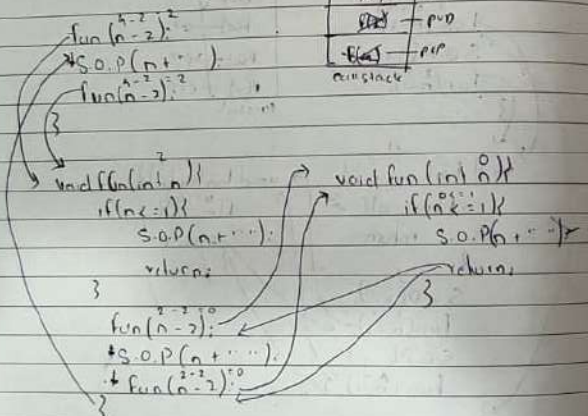
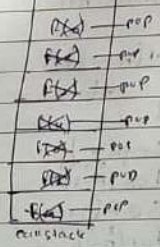
f(0) - pop

f(0) - pop

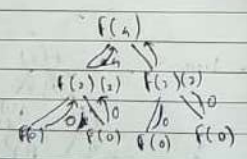
Q24 psum()

a/p:
 [0 2 0 5 0 2 0]

```
fun(h);
{
    void fun(int n) {
        if (n <= 1) {
            S.O.P(n + "...");
            return;
        }
    }
}
```



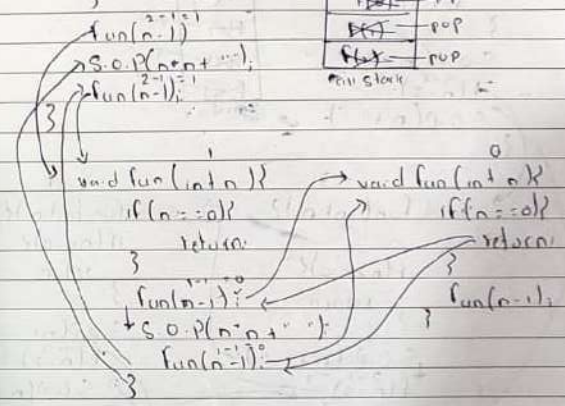
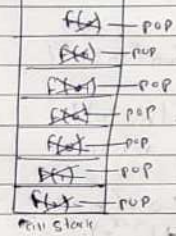
* Recursion Tree



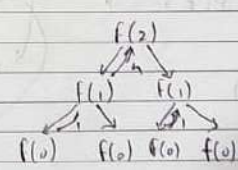
Q25 psum()

a/p:
 [1 1 1]

```
fun(2);
{
    void fun(int n) {
        if (n == 0) {
            return;
        }
    }
}
```



* recursion tree



Q26 psunf()

fun(s):

```

    void fun(int n) {
        if (n == 0) {
            return;
        }
        S.O.P(n + " ");
        f(n-2);
        S.O.P(n + " ");
    }

```

Call stack

```

    void fun(int n) {
        if (n == 0) {
            return;
        }
        S.O.P(n + " ");
        f(n-2);
        S.O.P(n + " ");
    }

```

```

    void fun(int n) {
        if (n == 0) {
            return;
        }
        S.O.P(n + " ");
        f(n-2);
        S.O.P(n + " ");
    }

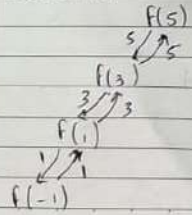
```

```

    void fun(int n) {
        if (n == 0) {
            return;
        }
    }

```

Recursion tree



olp: 5 3 1 1 3 5

Q27 psunf()

fun(8):

```

    void fun(int n) {
        if (n == 1) {
            S.O.P(n + " ");
            return;
        }
        S.O.P(n + " ");
        f(n/2);
        S.O.P(n + " ");
    }

```

Call stack

```

    void fun(int n) {
        if (n == 1) {
            S.O.P(n + " ");
            return;
        }
        S.O.P(n + " ");
        f(n/2);
        S.O.P(n + " ");
    }

```

```

    void fun(int n) {
        if (n == 1) {
            S.O.P(n + " ");
            return;
        }
        S.O.P(n + " ");
        f(n/2);
        S.O.P(n + " ");
    }

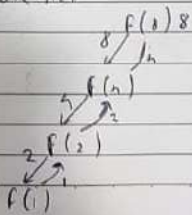
```

```

    void fun(int n) {
        if (n == 1) {
            S.O.P(n + " ");
            return;
        }
    }

```

Recursion tree



olp

8 4 2 1 2 4 8

Q28 psum(1)

fun(3)

```
void fun(int n) {
    if (n <= 0)
        return;
    S.O.P('A' + n + " ");
    if (n <= 1)
        S.O.P('B' + (n+2) + " ");
}
```

O/p:

A3 A2 A1 B2 B1

void fun(int n) {

if (n <= 0) return;

S.O.P('A' + n + " ");

if (n <= 1)

S.O.P('B' + (n+2) + " ");

void fun(int n) {

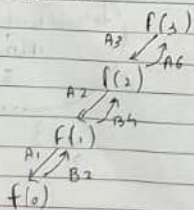
if (n <= 0) return;

S.O.P('A' + n + " ");

if (n <= 1)

S.O.P('B' + (n+2) + " ");

Recursion tree



Q29 psum(1)

fun(4)

void fun(int n) {

if (n <= 0) return;

fun(n-1);

S.O.P(n + " ");

fun(n-2);

void fun(int n) {

if (n <= 0) return;

fun(n-1);

S.O.P(n + " ");

fun(n-2);

void fun(int n) {

if (n <= 0) return;

fun(n-1);

S.O.P(n + " ");

fun(n-2);

void fun(int n) {

if (n <= 0) return;

fun(n-1);

S.O.P(n + " ");

fun(n-2);

void fun(int n) {

if (n <= 0) return;

fun(n-1);

S.O.P(n + " ");

fun(n-2);

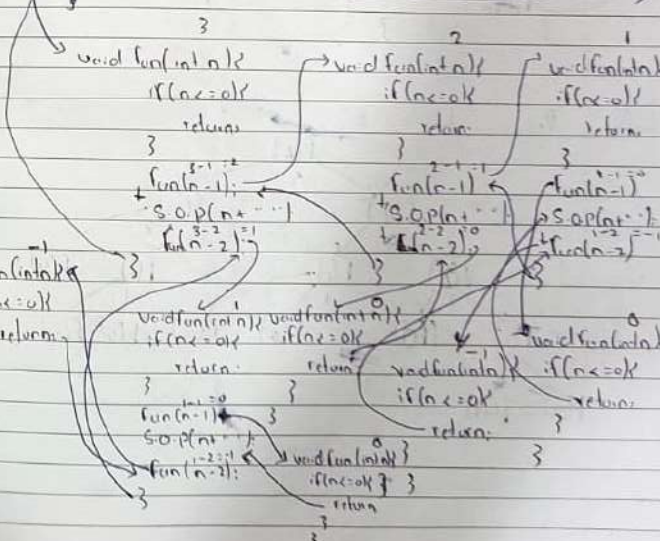
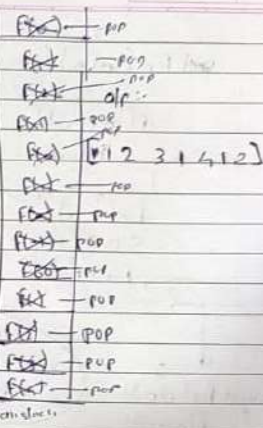
void fun(int n) {

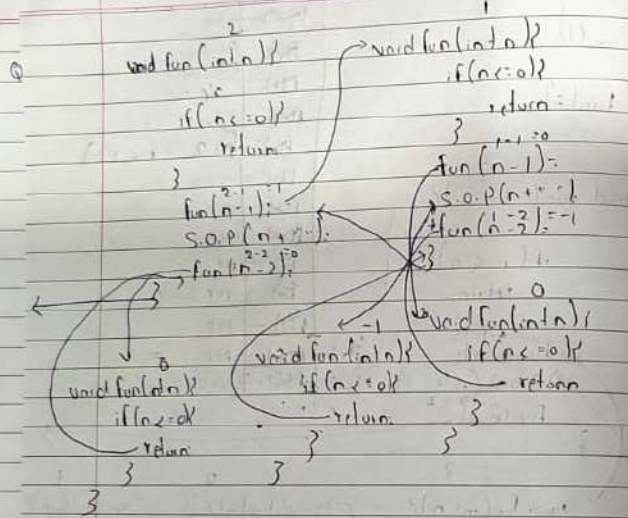
if (n <= 0) return;

fun(n-1);

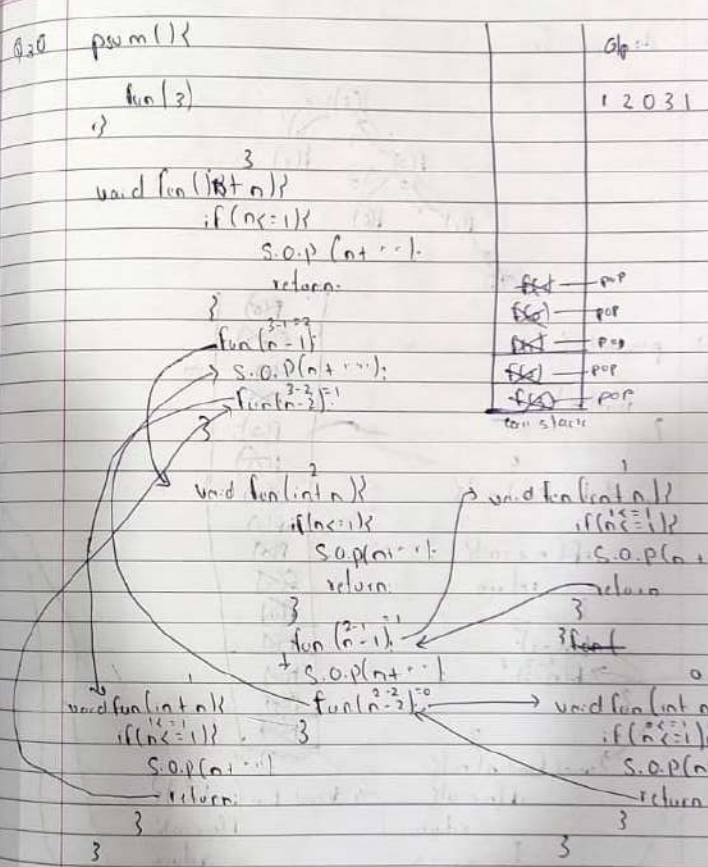
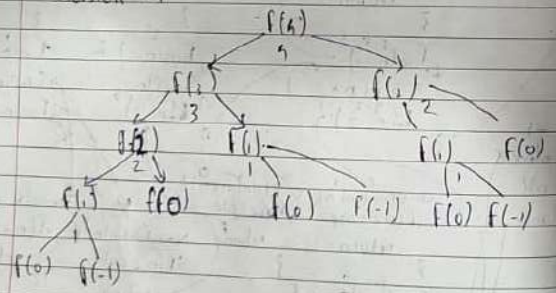
S.O.P(n + " ");

fun(n-2);





Recursion tree



Recursive Tree

Q 31 psum(1)

func(x);

void func(int n)

if (n <= 0)

return

n = n + 1

func(n-1);

func(n-2);

void func(int n)

if (n <= 0)

return

n = n + 1

func(n-1);

func(n-2);

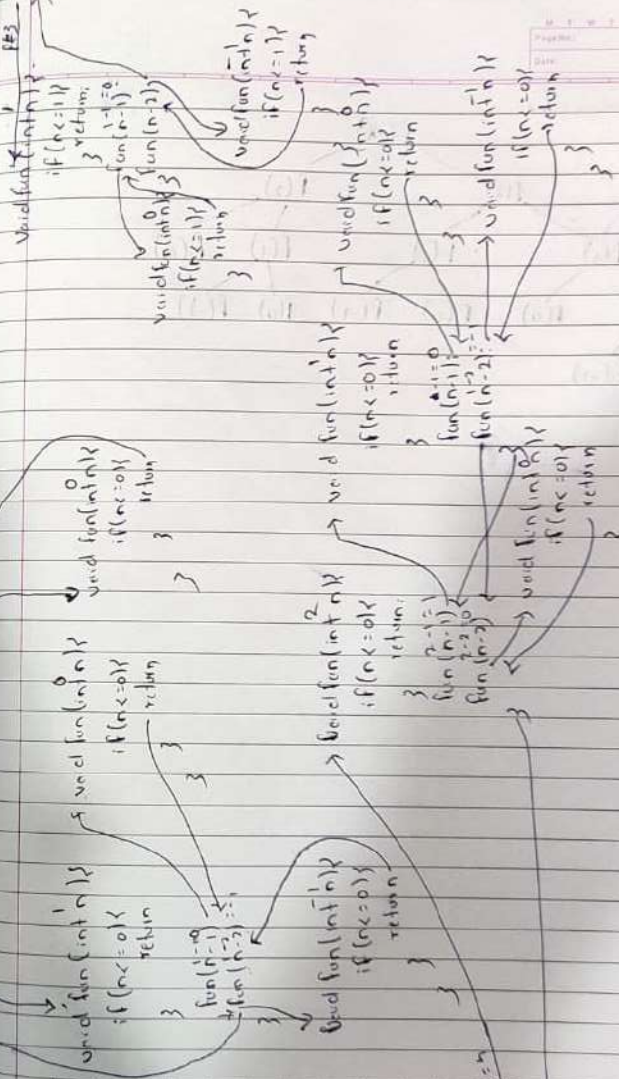
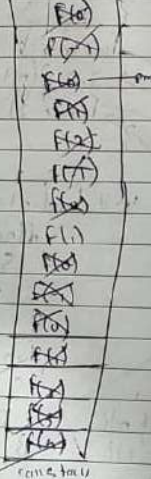
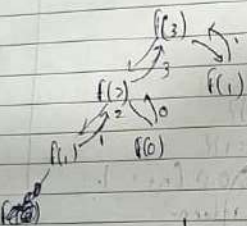
void func(int n)

if (n <= 0)

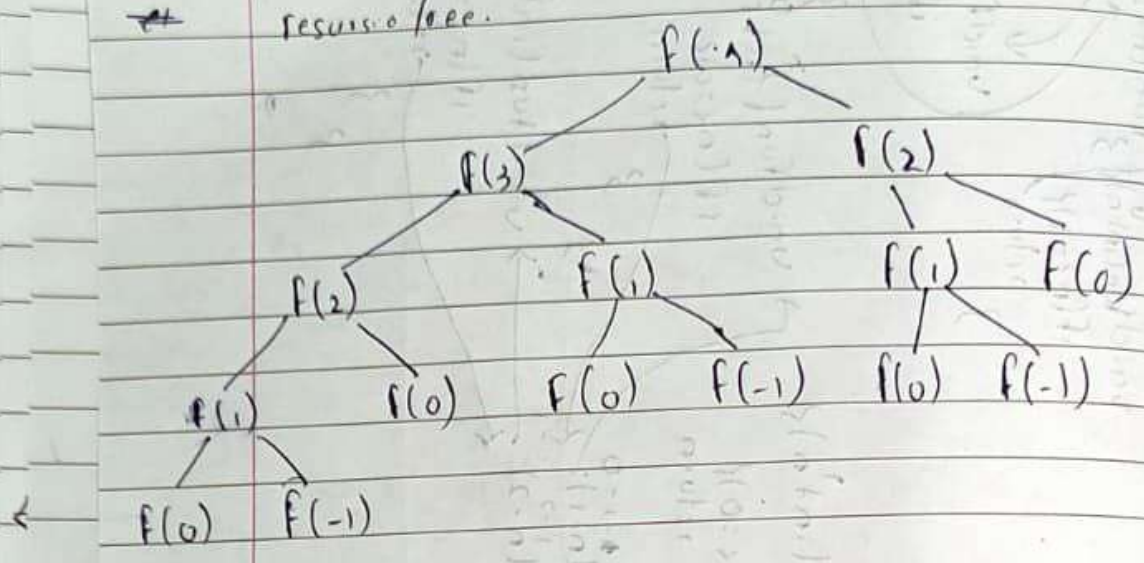
return

func(n-1);

func(n-2);



→ recursive Tree.



←

Q

←